

Agent评测体系深度拆解：从方法论到工程落地的完整路径

我最近在研究Agent评测这块，越深挖越觉得这玩意儿的复杂度远超我之前的认知。之前我以为评测就是跑个Benchmark、看个Pass Rate，但真正到落地阶段才发现——Benchmark只是冰山一角，真正的工程量在水面以下。

这篇不是一个"评测工具大全"或者"Benchmark列表"，而是我调研了美团图灵评测团队的分享、Anthropic的工程博客、牛客上的面经、以及自己踩过坑之后，整理出来的一套**从0到1搭建Agent评测体系**的完整路径。工程导向，讲HOW，讲落地，讲坑。

一、Agent评测为什么难

这部分简短说。核心一句话：**你评的不是模型，是"模型+系统+工具+流程"的复合系统。**

传统模型评测（MMLU、HumanEval那些），输入一个问题看输出对不对，确定性很强。Agent完全不同：

- **非确定性**：同一个任务跑十次，十条不同的执行路径，可能有七条路走通了但过程完全不同
- **路径多样性**：正确答案不唯一。你不能用标准答案去匹配，因为"走另一条路但也到了终点"同样是对的
- **错误累积**：链条很长，一步偏了后面全崩。单步准确率90%，串10步下来整体成功率可能只有35%

美团图灵团队在技术博客里有个总结我觉得说得特别到位——"ChatAgent评测关心'说得好不好'，长程Agent评测关心'事情做成没有，以及是怎么做成的'"。

这两个问题的评测方法论完全不同。前者还能用相似度、人工打分这些传统NLP手段，后者需要一整套新的基础设施。

二、评测体系的核心不是堆指标，而是"搭桥"

这是我调研过程中最大的认知升级。

很多团队做评测的方式是：列一堆指标、弄个打分表、让人来评。这种做法的问题是——你拿到一堆分数之后，不知道该优化什么，也不知道指标和业务目标之间是什么关系。

真正有效的评测体系，核心是在**业务指标和可操作的系统指标**之间搭一座桥。

三层指标桥

[复制代码](#)

业务层：转化率、用户留存、投诉率、任务完成率、NPS

↓ 归因拆解

任务层：意图识别准确率、信息抽取F1、计划执行完整性、结果正确性

↓ 技术定位

系统层：工具调用成功率、轨迹稳定性、端到端延迟、Token消耗、错误类型分布

举个例子——你的AI搜索Agent投诉率上升了。投诉率是业务层指标，你不能直接优化它。你需要拆到任务层：是意图识别错了（用户搜A你理解成B）？还是信息抽取漏了关键信息？还是结果排序有问题？然后再拆到系统层：是RAG检索召回不够？还是模型幻觉？还是某个数据源API超时？

每一个指标，你需要写清楚四件事：

要素	说明	示例
指标名称	唯一标识	tool_call_precision
判断对象	这个指标评的是什么	Agent选择的工具集合
判断规则	怎么算分	与Ground Truth工具集合的Precision
失败后动作	分数低了怎么办	触发工具选择模块的Case分析

美团的经验是：他们从最初20个指标经过1年Bad Case驱动的迭代，扩展到了200个指标。不是一开始就设计200个——那不现实，是每次线上出问题，分析根因之后发现"现有指标没覆盖到这种情况"，就新增一个。

三、如何从0搭建评测体系：完整路径

这是全文最重要的章节。我尽量把每个步骤讲到可直接操作的粒度。

3.1 第一步：设计评测维度

不同类型的Agent需要评测的维度不同，但有一套通用的7维度框架可以作为起点：

维度	评什么	适用场景	典型判断方法
意图识别	是否正确理解了用户的真实问题	所有Agent	分类匹配/LLM判断
数据正确性	引用的数据、事实是否准确	Data Agent / RAG Agent	与数据源核对
推理逻辑	分析过程是否合理、有无逻辑跳跃	分析类Agent	LLM-as-Judge
归因准确性	根因定位是否正确	诊断类Agent	与标注根因匹配
输出格式	结构是否符合预设模板	所有Agent	JSON Schema验证
建议质量	给出的建议是否具体、可执行	咨询类Agent	Rubric打分
合规性	是否包含敏感/违禁/越权内容	面向外部用户的Agent	规则+模型双检

设计维度的原则是MECE——互不重叠、完全覆盖、独立可评。如果两个维度的判断标准高度相关（比如“推理逻辑”和“归因准确性”在你的场景里几乎是同一回事），就合并成一个。

一个常见的坑是维度设计得太细。你搞了15个维度，每个维度都需要独立的评测逻辑、标注数据、Rubric——工程量翻了好几倍，但信息增量很小。实际经验是5-8个维度就够覆盖大部分场景了。

3.2 第二步：构建评测集

评测集是评测体系的“弹药”。设计不好，后面全白搭。

样本结构：

复制代码

```
{
  "id": "eval_001",
  "input": {
    "query": "帮我查一下上周五北京仓的出库量",
    "context": {"user_role": "运营", "current_date": "2026-08-20"}
  },
  "ground_truth": {
    "intent": "data_query",
    "key_entities": ["北京仓", "出库量", "2026-08-15"],
  }
}
```

```

    "expected_tool": "warehouse_query_api",
    "expected_result_contains": ["数值型出库量数据"]
  },
  "metadata": {
    "scenario": "数据查询",
    "difficulty": "easy",
    "source": "线上真实query",
    "created_at": "2026-08-01"
  }
}

```

关键点：

- **Input是真实Query**：不要自己编，从线上日志里捞。自己编的Query和真实用户的表达方式差距很大
- **Ground Truth只标必要字段**：不要试图标注"完美答案是什么样"。标注关键判据就够了——意图对不对、关键实体提取了没有、该调的工具调了没有
- **元数据要打场景标签和难度标签**：后面分析时按场景/难度切片看效果，能发现很多insight

覆盖策略——场景×难度矩阵：

	Easy	Medium	Hard	Edge Case
数据查询	10条	10条	5条	5条
任务执行	10条	10条	5条	5条
多轮对话	10条	10条	5条	5条
异常处理	5条	5条	5条	5条

核心场景50-200条就够起步了。不需要一上来搞几千条。少而精远比多而糙有价值。

Ground Truth标注原则：

1. 只标必要字段——你后面的评分规则用不到的字段就别标，浪费标注成本
2. 标注一致性用Cohen's Kappa衡量——两个标注员独立标注同一批数据，Kappa > 0.8才说明标注规则足够清晰
3. 持续迭代——每次发现评测有False Positive/Negative，回头看是标注问题还是评分规则问题

冷启动建议：

如果你的Agent刚开始做、没有线上数据，建议用10-30条种子样本启动：

- 5-10条预期成功样本（Happy Path）
- 5-10条预期失败/困难样本（边界条件、模糊指令）
- 3-5条权限边界样本（Agent应该拒绝的请求）
- 3-5条异常输入（空字符串、超长文本、注入攻击）

3.3 第三步：设计Rubric（评分规则）——关键中的关键

这一步是整个体系里最容易做烂的环节。

错误示范：

"请判断大模型回答是否口语化，按0-10分打分"

这种Rubric给三个人评，三个人打出来的分数能差5分。"口语化"这个概念太模糊了，每个人理解不同。

正确做法——拆成二元判断项：

把一个模糊的维度拆成多个明确的Yes/No/Unknown判断：

∨ 复制代码

```
dimension: 口语化程度
sub_criteria:
  - id: c1
    question: "模型是否以'您'指代用户？"
    options: [是, 否, 未知]
    positive_example: "您好, 您的订单已发货"
    negative_example: "用户的订单已完成发货操作"

  - id: c2
    question: "模型输出是否使用口语词汇（如'搞定''OK''没问题'）？"
    options: [是, 否, 未知]
    positive_example: "搞定了, 您看一下对不对"
    negative_example: "处理完毕, 请确认结果"

  - id: c3
    question: "模型输出是否包含语气词（如'哈''呢''呀'）？"
    options: [是, 否, 未知]
    positive_example: "这个问题比较复杂呢"
    negative_example: "该问题较为复杂"

scoring_rule: "三项中≥2项为'是', 则该维度通过"
```

这样设计之后，不同的评测员（不管是人还是LLM）对同一个样本的打分一致性会大幅提升。

Rubric至少包含5个要素：

要素	说明
判断对象	具体看Agent输出的哪个部分
正例和反例	各2-3个，消除歧义
允许的证据范围	判断时可以参考什么信息（只看输出？还是也看过程？）
不确定时的处理方式	无法判断时标Unknown，而不是强行选一个
与业务目标的关系	这个子项通过/不通过，对业务的影响是什么

Rubric验收标准：

验收项	达标标准	不达标时的处理
标注员间一致率	Cohen's Kappa ≥ 0.8	回头改Rubric定义
Unknown占比	$< 10\%$	Rubric定义不够清晰，需要补充边界案例
与业务指标相关性	Rubric得分与线上表现正相关	说明评测维度选错了
覆盖度	线上Bad Case $\geq 80\%$ 能被某个子项捕获	需要新增子项

3.4 第四步：实现评分机制（三类Grader详解）

有了Rubric之后，需要把它落地成可执行的评分代码。根据评分逻辑的复杂度，分三类Grader。

确定性规则Grader

能用确定性规则判断的，绝不用LLM。因为确定性规则100%可复现、0成本、极快。

JSON Schema验证：

∨

复制代码

小红书

小红书号:49329444782

```

import jsonschema

expected_schema = {
    "type": "object",
    "required": ["action", "parameters"],
    "properties": {
        "action": {"type": "string", "enum": ["query", "create",
"update"]},
        "parameters": {"type": "object"}
    }
}

def grade_format(agent_output):
    try:
        jsonschema.validate(agent_output, expected_schema)
        return {"passed": True, "score": 1.0}
    except jsonschema.ValidationError as e:
        return {"passed": False, "score": 0.0, "reason": str(e)}

```

单元测试执行（代码Agent场景）：

Agent生成的代码能不能跑通测试——这是最硬的评测。SWE-bench用的就是这个思路。

工具调用验证：

这个要特别展开说，因为工具调用是Agent最核心的能力之一。评测工具调用需要看三个子指标：

 复制代码

```

def evaluate_tool_calls(predicted_calls, expected_calls):
    pred_set = set(predicted_calls)
    exp_set = set(expected_calls)

    precision = len(pred_set & exp_set) / len(pred_set) if pred_set
else 0
    recall = len(pred_set & exp_set) / len(exp_set) if exp_set else 0

    # 还要检查参数准确率

    param_scores = [ ]

    for call in pred_set & exp_set:
        pred_params = get_params(call, predicted_calls)
        exp_params = get_params(call, expected_calls)
        param_scores.append(compare_params(pred_params, exp_params))

```

```

return {
    "tool_precision": precision,      # 调了的工具里多少是该调的
    "tool_recall": recall,            # 该调的工具里多少被调了
    "param_accuracy": mean(param_scores), # 参数填对了多少
    "invalid_calls": len(pred_set - exp_set), # 多余的无效调用
}

```

LLM-as-Judge Grader

对于无法用规则判断的维度（推理质量、建议可执行性等），用LLM来当裁判。

评测Prompt的结构（这个结构是经过大量实验验证的）：



复制代码

```

system: |
    你是一个专业的Agent输出质量评测员。
    你的任务是根据给定的评分标准，对Agent的输出进行评分。
    请严格按照评分标准执行，不要引入个人偏好。

user: |
    ## 评测目标
    评估Agent对用户查询的回答是否准确且有帮助。

    ## 评分标准
    - 5分：回答完全正确，包含所有关键信息，表述清晰
    - 4分：回答基本正确，包含主要关键信息，有微小遗漏
    - 3分：回答方向正确，但有明显信息遗漏或不准确
    - 2分：回答部分正确，但存在关键性错误
    - 1分：回答基本错误或完全无关

    ## 关键判据（Ground Truth）
    {ground_truth}

    ## 用户原始查询
    {user_query}

    ## Agent输出
    {agent_output}

    ## 请输出
    ```json
 {
 "score": <1-5>,
 "is_critical_error": <true/false>,

```

```
"reasoning": "<判断依据，引用具体内容>",
"missed_points": ["<遗漏的关键点>"]
}
```

[复制代码](#)

**\*\*`is\_critical` 标记: \*\*** 这个很重要。区分"瑕疵"和"致命错误"。比如客服Agent告诉用户"可以退款"但实际不满足退款条件——这是致命错误，不管其他方面多好，这条直接判失败。而"回复稍微啰嗦了点"只是瑕疵，不影响大局。

**\*\*LLM Judge的校准方法: \*\***

1. 先准备20-30条"黄金标准"样本，人工打好分
2. 用LLM Judge跑一遍这批样本
3. 对比人机一致率:
  - 一致率  $\geq 80\%$ : Judge可用
  - 一致率 60-80%: 需要优化Prompt或Rubric
  - 一致率  $< 60\%$ : 重新设计评测方案

**\*\*LLM Judge的已知偏见和缓解方法: \*\***

偏见类型	表现	缓解方法
-----	-----	-----
位置偏见	倾向于选择排在前面的候选	交换候选顺序，取两次结果的共识
长度偏见	倾向于给更长的回答更高分	Rubric里明确说"长度不是加分项"
文风偏见	偏好与自身生成风格相似的文本	盲化候选来源
自我偏好	同一模型当Judge时偏好自己生成的内容	用不同模型当Judge

Anthropic在工程博客里提出了一个实操建议：用多个不同的LLM当Judge，取多数投票结果。如果三个Judge中有两个给了相同的分，就采信那个分数。如果三个Judge各说各话，这条样本标记为"需人工复审"。

#### #### 人工评分Grader

有些维度LLM也搞不定，必须要人。比如"这个建议在实际业务中是否真的可执行"，需要领域专家来判断。

人工评分的对齐流程：

核心评测员对齐（2-3人，背靠背标注→对齐→形成标准）



外包对齐（核心评测员带外包团队，确保一致率  $> 85\%$ ）

小红书

小红书号:49329444782



机评对齐（用对齐好的人工标注结果去校准LLM Judge）



复制代码

"背靠背标注"是指：两个评测员独立标注同一批数据，然后对比分歧。分歧多的地方说明Rubric定义不清楚，需要修改。反复这个过程直到一致率达标。

### ### 3.5 第五步：自动化执行管线

前面几步都是"设计"层面的，这一步是"工程"层面的——把评测跑起来。

**\*\*批量评测流程：\*\***

数据源(评测集JSON)

- 并发调用Agent(控制QPS，避免打爆下游)
- 收集Agent输出(含完整Trace)
- 按维度执行评分(规则Grader + LLM Grader)
- 汇总得分
- 生成报告(按场景/难度/维度多维切片)



复制代码

**\*\*工程要点：\*\***

```
```python
```

```
# 伪代码示意评测管线核心逻辑
```

```
class EvalPipeline:
```

```
    def __init__(self, config):
```

```
        self.max_concurrency = config.get("max_concurrency", 5)
```

```
        self.retry_limit = config.get("retry_limit", 3)
```

```
        self.checkpoint_dir = config.get("checkpoint_dir",  
". / checkpoints")
```

```
    async def run(self, eval_set):
```

```
        results = self.load_checkpoint() # 断点续跑
```

```
        pending = [item for item in eval_set if item.id not in results]
```

```
        semaphore = asyncio.Semaphore(self.max_concurrency)
```

```
        tasks = [self.eval_one(item, semaphore) for item in pending]
```

```
        for completed in asyncio.as_completed(tasks):
```

小红书

小红书号: 49329444782

```

        result = await completed
        results[result.id] = result
        if len(results) % 10 == 0:
            self.save_checkpoint(results) # 定期存盘

    return self.generate_report(results)

    async def eval_one(self, item, semaphore):
        async with semaphore:
            for attempt in range(self.retry_limit):
                try:
                    agent_output = await self.call_agent(item.input)
                    scores = await self.grade(agent_output,
item.ground_truth)
                    return EvalResult(id=item.id, output=agent_output,
scores=scores)
                except Exception as e:
                    if attempt == self.retry_limit - 1:
                        return EvalResult(id=item.id, error=str(e))

```

几个关键点：

- **并发控制**：不要无限并发调用Agent，会把下游打崩。5-10的并发度够了
- **断点续跑**：评测可能跑几个小时，中间挂了别从头来。每跑完一条就存一条
- **容错处理**：Agent可能超时、返回异常格式、甚至直接报错。都要graceful处理
- **结果持久化**：每次评测的完整结果（输入、输出、Trace、分数）都存下来，后面分析用

触发机制：

- 手动触发：开发过程中随时跑
- 定时触发：每天凌晨跑一次完整回归
- CI/CD集成：Agent代码有改动时自动触发

四、人人一致 & 人机一致：评测对齐方法论

这部分我主要参考的是美团图灵评测团队的分享，他们在这块踩的坑非常多，总结的经验很实在。

"独裁者"制度

这个名字听起来极端，但实操中非常有效。核心思想：**1个独裁者好过10个民主者**。

你让10个人投票决定一条回答是3分还是4分，结果可能5:5。但如果有一个"独裁者"（通常是最资深、对业务理解最深的那个人），他说"这条就是3分，原因是XXX"——标准就定下来

小红书

小红书号:49329444782

了。其他所有人以他为锚点对齐。

这不是说独裁者不能被质疑。而是说在对齐过程中需要一个唯一的参考点，否则标准永远统一不了。

人人一致的对齐流程



复制代码

- Step 1: 背靠背标注
- 2-3个核心评测员独立标注同一批50条样本
 - 不交流、不参考彼此的结果
- Step 2: 汇总分歧
- 把不一致的样本拎出来
 - 逐条讨论分歧原因
- Step 3: 修改Rubric
- 分歧的根因通常是Rubric定义不够清晰
 - 补充边界案例、修改判断规则
- Step 4: 重新抽样验证
- 用新Rubric重新标注一批新数据
 - 一致率 $\geq 85\%$ 则通过，否则回到Step 2

人机一致的对齐流程

人和对齐了之后，下一步是让LLM Judge和人对齐。



复制代码

- Step 1: 指标下钻
- 把模糊指标拆成二元子项（参考3.3的Rubric设计）
- Step 2: 用Unknown占比反查定义质量
- 如果LLM Judge大量标Unknown，说明Rubric定义不够机器可理解
 - 需要补充更多正例反例
- Step 3: 迭代直到一致率 $> 85\%$
- 每轮对比人机标注结果
 - 分析分歧Case，优化Judge Prompt
- Step 4: 置信度标记
- 一致率 $> 95\%$ 的维度：完全自动化
 - 一致率 85-95%的维度：自动化+低置信度抽检

- 一致率 < 85%的维度：人工为主

美团公开的数据：他们的"数字站长"Agent，人机一致率做到了99%。另一个产品线Beam从最初的62%提升到了92%——提升过程就是不断优化Rubric和Judge Prompt的过程。

五、不同类型Agent的评测策略

Anthropic在工程博客里把Agent分成了四类，每类的评测方法差异很大。我把他们的实践整理出来。

编码Agent

评测要素：

- **单元测试通过率**：最硬的指标。Agent写的代码能不能过测试
- **静态分析**：有没有引入新的lint错误、类型错误
- **LLM代码质量评分**：代码风格、可读性、是否过度工程
- **工具调用验证**：文件读写、命令执行是否合理

Anthropic的Task定义示例（编码Agent）：



复制代码

```
task:
  name: "fix_auth_middleware_bug"
  prompt: |
    The authentication middleware in src/middleware/auth.ts is not
    properly validating expired tokens. Users with expired tokens
    can still access protected routes. Fix this bug.

  repository: "github.com/example/web-app"
  branch: "main"

  expected_behavior:
    - description: "Expired tokens should be rejected"
      grader: "unit_test"
      test_file: "tests/auth.test.ts"
      test_name: "rejects expired tokens"

    - description: "Valid tokens should still work"
      grader: "unit_test"
      test_file: "tests/auth.test.ts"
      test_name: "accepts valid tokens"
```

小红书

```

- description: "No unnecessary file changes"
  grader: "deterministic"
  rule: "changed_files_count <= 3"

- description: "Code quality maintained"
  grader: "llm"
  rubric: "No reduction in code clarity or introduction of code
smells"

trials: 5
timeout_seconds: 300

```

对话Agent（客服/助手）

对话Agent的评测复杂度在于——你不能只评单轮回答质量，还要评多轮交互的整体体验。

Anthropic的做法是用第二个LLM模拟用户，和Agent进行多轮对话，然后评测整个对话过程。

 复制代码

```

task:
  name: "handle_refund_request"
  prompt: |
    You are a customer service agent for an e-commerce platform.
    Handle the customer's refund request according to company policy.

  simulated_user:
    persona: "Frustrated customer whose order arrived damaged"
    goal: "Get a full refund for order #12345"
    behavior: "Initially emotional, becomes cooperative when treated
well"

  environment:
    order_database:
      order_12345: {status: "delivered", amount: 299, days_since: 3}
    refund_policy:
      - "Full refund within 7 days for damaged items"
      - "Must collect photo evidence before processing"

  expected_behavior:
    - description: "Correctly identifies refund eligibility"
      grader: "deterministic"
      rule: "agent_confirms_eligibility == true"

```

```
- description: "Requests photo evidence"
  grader: "trajectory"
  rule: "agent_requests_photo in conversation"

- description: "Maintains professional tone throughout"
  grader: "llm"
  rubric: "No dismissive language, acknowledges frustration"

- description: "Resolves within reasonable turns"
  grader: "deterministic"
  rule: "total_turns <= 8"

- description: "Final state correct"
  grader: "state_check"
  rule: "refund_initiated == true AND amount == 299"

max_turns: 15
trials: 5
```

研究Agent

研究Agent（帮你查资料、写报告的）评测最难，因为“好的研究”本身就没有标准答案。

Anthropic的建议是分层评：

- **基础性检查（确定性）**：引用的来源是否真实存在、数据是否正确
- **覆盖度（LLM Judge）**：关键方面是否都覆盖到了
- **来源质量（LLM+人工）**：引用的来源是否权威、是否过时

研究Agent需要LLM和人工混合评测，纯自动化目前做不到位。

计算机操作Agent

评测计算机操作Agent（点网页、操作桌面的）需要真实或沙盒环境。核心检查的是**环境终态**——Agent操作完之后，屏幕上/系统里的状态是否符合预期。

两种评测方式：

- **DOM检查**：直接检查页面元素状态。精确但不是所有环境都能拿到DOM
- **截图比对**：用视觉模型比对截图。更通用但精度略低

六、长程Agent评测新范式

这部分是评测领域最前沿的方向，也是最难的。

短程vs长程

维度	短程Agent	长程Agent
交互模式	Query → Answer	多步骤任务执行
评测对象	单次输出质量	整个执行轨迹+最终结果
正确答案	通常唯一	多条有效路径
典型时长	秒级	分钟到小时级
评测难度	中等	极高

Trajectory Evaluation

长程Agent评测的核心概念是三元组：**(prompt, expected_behavior, trace)**

- **prompt**: 给Agent的任务描述
- **expected_behavior**: 期望Agent满足的条件集合（不是唯一正确答案）
- **trace**: Agent实际的执行轨迹（调了哪些工具、每步的输入输出、中间状态）

评测时不是检查"Agent走的路径和标准路径一样"，而是检查"Agent走的路径是否满足所有必要条件"。

比如一个文件操作任务的expected_behavior可以这样定义：



复制代码

```
expected_behavior:
  must_do:
    - "Read config.yaml before making changes"
    - "Create backup before modifying production files"
    - "Validate output format after generation"
  must_not_do:
    - "Delete any file without confirmation"
    - "Access files outside workspace directory"
  final_state:
    - "output.json exists and is valid JSON"
    - "output.json contains field 'status' with value 'completed'"
  budget:
    max_steps: 20
    max_tool_calls: 15
    max_tokens: 50000
```

Skill评测的三大痛点

美团在做长程Agent评测时总结了三个核心痛点：

1. **Skill交叉干扰**：Agent有多个Skill（比如"查数据"、"发通知"、"写报告"），评测时很难把一个Skill的效果和另一个Skill的影响完全隔离开
2. **环境依赖**：很多Skill依赖外部环境（数据库状态、第三方API），环境不稳定导致评测不可复现
3. **评测粒度**：太粗（只看最终结果）发现不了问题；太细（检查每一步）容易误杀合理的替代路径

从"人评主导"走向"机评主导"

美团分享了他们的演进路线：

复制代码

阶段1（2024初）：纯人工评测。10个人评50条Case，一天干完

↓

阶段2（2024中）：人工出标准 + LLM批量执行。效率提升10x

↓

阶段3（2025）：LLM评测为主 + 人工抽检校准。覆盖面从50条扩大到2000条

↓

阶段4（2026）：全自动管线 + 异常触发人工。日常完全自动化

核心公式：**观测 + 评测 = 持续迭代**。线上观测发现问题 → 分析归因 → 补充评测Case → 优化Agent → 再观测。这个循环不停转。

七、评测集的持续运营（数据飞轮）

评测不是"做一次就完了"的事。评测集本身需要持续运营。

五个关键环节

复制代码

采集：线上日志/用户反馈/主动探索

- 清洗：去重、脱敏、格式标准化
- 评测：跑管线出分数
- 质检：抽检评分结果的准确性
- 分析/归因：按维度/场景分析薄弱点
- 回到采集（补充新Case）

Bad Case的价值高于Good Case

这是一个反直觉的点。很多人觉得评测集应该"均匀覆盖各种情况"，但实际上——一条Bad Case的信息量远高于十条Good Case。

Good Case告诉你"Agent能做对这种事"。这很好但不够。Bad Case告诉你"Agent在这种情况下会犯这种错"——这才是优化的方向。

最佳实践路径：



复制代码

高频核心场景入手（覆盖80%流量的场景）

- 线上收集Bad Case（实际失败的case）
- 沉淀Good Case（修复后的成功版本）
- 转标准样本（加Ground Truth、打标签）
- 反哺优化（优化Agent后验证是否修复）
- 持续迭代（新的Bad Case不断补充进来）

种子评测集与冷启动策略

刚开始没有线上数据的时候：

1. **专家构造**：让最了解业务的人手写10-20条核心case。包括"肯定应该成功的"和"肯定应该失败的"
2. **合成扩充**：用LLM基于种子case生成变体（改措辞、换参数、增加干扰条件）
3. **Adversarial补充**：刻意构造Agent容易犯错的case——边界条件、模糊指令、陷阱题
4. **上线后真实替换**：一旦有了线上流量，逐步用真实case替换合成case

八、Agent评测在面试中怎么考

我翻了牛客和一些面经帖子，整理了评测相关的高频问题和回答框架。

Q1: Agent最核心的评测指标是什么？

不要只说"任务完成率"。完整的回答框架：

- **任务完成率**（按类型和难度分层看，总体的任务完成率意义不大）
- **正确性**（完成了但结果是错的也不行）
- **工具选择准确性**（选对了工具、参数填对了）
- **步骤效率**（用了多少步完成，有没有冗余操作）
- **延迟**（端到端耗时）
- **成本**（Token消耗、API调用次数）

- **安全合规**（有没有越权、泄露敏感信息）
- **用户体验**（交互是否自然，有没有死循环或反复确认）

关键点：这些指标不是平权的。在不同业务场景里权重完全不同。金融场景里正确性和合规性权重极高，效率Agent里延迟和成本是核心。

Q2: Outcome Eval vs Trajectory Eval?

这是一个高频问题。答案框架：

- **Outcome Eval**：只检查终态。任务完成了没有？结果对不对？——简单、低成本、但无法归因
- **Trajectory Eval**：检查执行过程。路径合理吗？有没有多余步骤？有没有危险操作？——复杂、高成本、但能精确定位问题

什么时候用什么：

- 结果可验证（如代码能跑测试、数据库状态可查）：优先Outcome
- 安全合规要求高（如金融、医疗）：**必须**检查Trajectory。即使结果对了，过程中有越权操作也不行
- 需要优化Agent的执行策略：Trajectory提供优化方向

两者不是二选一，而是互补的。

Q3: Ground Truth路径不唯一怎么办？

这个问题问的是"Agent走了一条和标准答案不同的路，但结果也是对的，怎么评？"

答案：**不强制唯一动作序列**。改为定义约束集合：

```
evaluation_constraints:
  allowed_tools: ["search_api", "database_query", "calculator"]
  preconditions:
    - "Must verify user identity before accessing personal data"
  invariants:
    - "Never expose raw SQL to user"
    - "Total API calls <= 10"
  final_state:
    - "User receives correct answer"
    - "Response time < 30s"
  budget:
    max_steps: 15
    max_tokens: 30000
```

复制代码

只要Agent在约束范围内完成了任务，不管走的是哪条路，都算通过。

Q4: 如何评测工具使用？

完整的工具使用评测维度：

子指标	含义
Precision	调了的工具里，多少是该调的
Recall	该调的工具里，多少被调了
参数准确率	工具选对了，参数填对了多少
无效调用	调了但没用上返回结果的（浪费）
重复调用	同样参数调了多次（可能是Agent陷入循环）
必要工具遗漏	有个关键工具该调但没调
结果利用率	调用返回的结果，Agent在后续步骤中用了多少

Q5: 离线vs在线评测怎样配合？

维度	离线评测	在线评测
速度	快，可批量跑	慢，依赖真实流量
可复现	高	低（环境变化）
分布	评测集分布（可能有偏）	真实分布
成本	低	高
用途	回归验证、比较方案	真实效果验证

发布流程应该是：

✓

复制代码

单元测试（工具调用、格式验证等硬性检查）
→ 离线回归（跑完整评测集，对比baseline）
→ 安全红队（专门的攻击性测试）



- 小流量AB（1-5%真实流量验证）
- 扩量发布

牛客上见到的网易面试真题

Q: Agent测试主要关注哪些方面？

回答要点：功能正确性（任务完成）、稳定性（多次执行一致性）、安全性（越权/泄露）、效率（延迟/成本）、异常处理（工具失败时的fallback）。

Q: Skill命中率的自动化评测怎么做？

回答：

1. 构建意图分类评测集（Query → 期望命中的Skill）
2. 批量调用Agent的路由模块
3. 对比实际命中Skill vs 期望Skill
4. 计算分场景的Precision/Recall
5. 持续用线上misroute case补充评测集

Q: 用户反馈"非预期内容"怎么定位？

回答：

1. 拿到用户反馈的具体case
2. 回放Agent的完整Trace（每步的输入输出）
3. 逐步检查哪一步开始偏离预期
4. 分类归因：是意图识别错了？工具选错了？参数填错了？还是结果处理有问题？
5. 针对根因类别补充对应的评测case

九、可复用的评测体系架构设计

做到一定阶段，你会面临一个问题：公司有多多个Agent（客服Agent、数据Agent、运营Agent...），每个都从零搭评测体系太浪费了。需要设计可复用的架构。

四层分层架构



复制代码

认知与方法论层（顶层复用）
三层指标桥、MECE维度设计、Rubric方法论
通用框架层（工程完全复用）

小红书

小红书号:49329444782



- **认知与方法论层**：全公司通用。怎么设计指标桥、怎么写Rubric、怎么做对齐——这些方法论一次搞清楚，所有Agent团队共用
- **通用框架层**：工程完全复用。评测管线的代码、并发控制、断点续跑、报告模板——写一套所有Agent用
- **领域配置层**：不同Agent插拔配置。客服Agent评"口语化"和"策略遵循"，数据Agent评"数据正确性"和"SQL安全"——配置不同但框架相同
- **评测数据层**：完全业务独有。每个Agent自己的评测集和Ground Truth

7维度的跨场景通用性示例

维度	客服Agent	数据Agent	编码Agent
意图识别	用户想退货/换货/咨询	用户想查什么数据	用户想修什么bug
数据正确性	订单信息是否正确	查询结果是否准确	代码逻辑是否正确
输出格式	是否符合话术模板	表格/图表格式是否规范	代码是否通过lint
合规性	是否泄露用户隐私	是否暴露敏感数据	是否引入安全漏洞

维度是通用的抽象，具体的判断标准按业务配置。

十、核心工具和框架推荐

Anthropic推荐的评测工具

Anthropic在博客里提到他们自己在用Promptfoo，同时也推荐了几个：

工具	特点	适用场景
Promptfoo	开源，Anthropic自己在用。 CLI驱动，支持多种Provider	快速迭代Prompt和评测
Harbor	开源Agent评测框架	专注Agent场景
Braintrust	企业级，精细化管道	大团队、复杂流程
LangSmith	LangChain生态	LangChain用户
Langfuse	开源，追踪+评测一体化	需要Observability的团队

选型建议

- 如果你是个人或小组刚起步：Promptfoo。开源、轻量、CLI直接跑
- 如果你用LangChain：LangSmith是无缝集成的
- 如果你需要全链路追踪：Langfuse。开源、self-hosted、隐私友好
- 如果你是大企业有复杂需求：Braintrust或自建

开源评测Benchmark

如果你想把自己的Agent和业界SOTA对比：

Benchmark	场景	特点
SWE-bench Pro	代码Agent	1865个真实GitHub issue
Claw-Eval	长程Agent	300任务，Pass@3评分，高难度
WildClaw-Bench	通用Agent	动态更新，抗数据污染
τ -bench	客服Agent	评策略合规
OSWorld	操作系统Agent	真实桌面环境

十一、收尾

Agent评测这件事，本质上正在从一个"打分动作"演变为一项"基础设施能力"。

几个我在调研过程中形成的核心判断：

评测对象变了——从"评一次回答好不好"变成"评一个任务系统能不能稳定完成工作"。这意味着你需要的不是一个打分函数，而是一整套包含数据管理、执行管线、质量校准、持续运营的基础设施。

评测方法变了——从Query→Answer的简单匹配，走向Prompt→Expected Behavior→Trace的复杂验证。这对工程能力的要求高了一个数量级。

评测的价值定位变了——评测不再只是"上线前验一下"，而是整个Agent开发循环的驱动力。Anthropic的Eval-Driven Development、美团的观测+评测双循环，都是把评测放在了开发流程的核心位置。

最后说一句：评测是一门实践科学，不是一蹴而就的。我见过太多团队花两周设计了一套"完美"的评测体系，然后发现实际跑起来跟想的完全不一样。正确的姿势是：先搞个能跑通的最小版本，然后在实战中不断打补丁。Bad Case会告诉你下一步该补什么。

核心参考资料

来源	内容	链接
Anthropic Engineering Blog	"Evaluations for AI agents" — Agent评测方法论完整版	anthropic.com/engineering/evaluations-for-ai-agents
美团技术博客	"Agent评测漫谈" — 国内大厂最详细的评测落地经验	tech.meituan.com
IBM+耶鲁 ACL 2026 Survey	"A Survey on Evaluation of LLM-based Agents" — 学术综述	arxiv.org
Claw-Eval (北大)	长程Agent评测，Pass ^k 指标	claw-eval.github.io
Sierra τ -bench	对话Agent+策略合规评测	sierra.ai
Promptfoo	开源评测框架，Anthropic推荐	promptfoo.dev
Langfuse	开源LLM Observability+评测	langfuse.com
牛客面经	Agent评测相关面试题	nowcoder.com
SWE-bench	代码Agent评测标杆	swebench.com

OSWorld (NeurIPS 2024)

操作系统Agent评测

github.com/xlang-ai/osworld